GEBRUIKSAANWIJZING

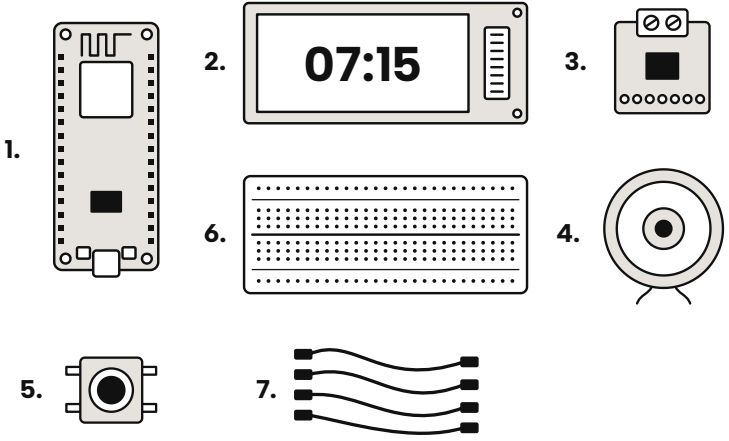


Gemaakt door: Daan Ravensbergen
Studentennummer: 500940161

Versie 1

Verschillende onderdelen

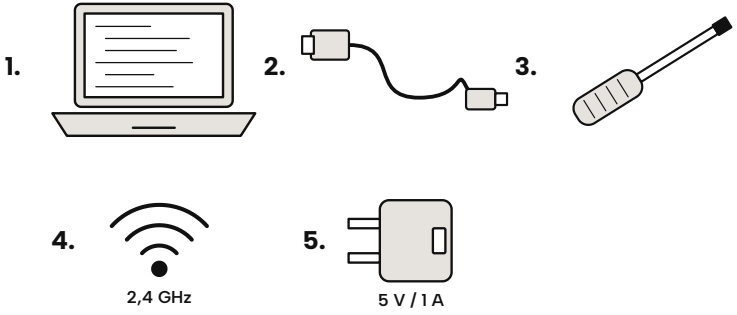
*niet op ware grootte afgebeeld



1. NodeMCU V3 (ESP8266)
2. E-ink-scherm 2,9 inch
3. Versterker MAX98357A
4. Speaker 4 ohm, 3 W
5. Drukknop
6. Breadboard
7. Jumperdraden male-female

Benodigdheden

*niet op ware grootte afgebeeld



1. Laptop met Arduino IDE
2. Micro-USB-kabel met data
3. Kleine schroevendraaier
4. Wifi met naam en wachtwoord
5. USB-lader van 5 V, 1 A of meer

STAP 1

1.1 Alles klaarleggen

Voordat je begint met bouwen, leg je alle onderdelen en benodigdheden van de vorige twee pagina's klaar. Je bouwt de wekker eerst los op tafel. Zo kun je alles testen voordat het in een behuizing gaat.

De code voor de wekker staat al klaar, achterin deze gebruiksaanwijzing. Maak in de Arduino IDE een nieuwe sketch met de naam *SlimmeWekker* en plak de code erin. Heb je het bestand `SlimmeWekker.ino`, zet het dan in een map met dezelfde naam: `SlimmeWekker/SlimmeWekker.ino`. Anders opent de Arduino IDE het niet.

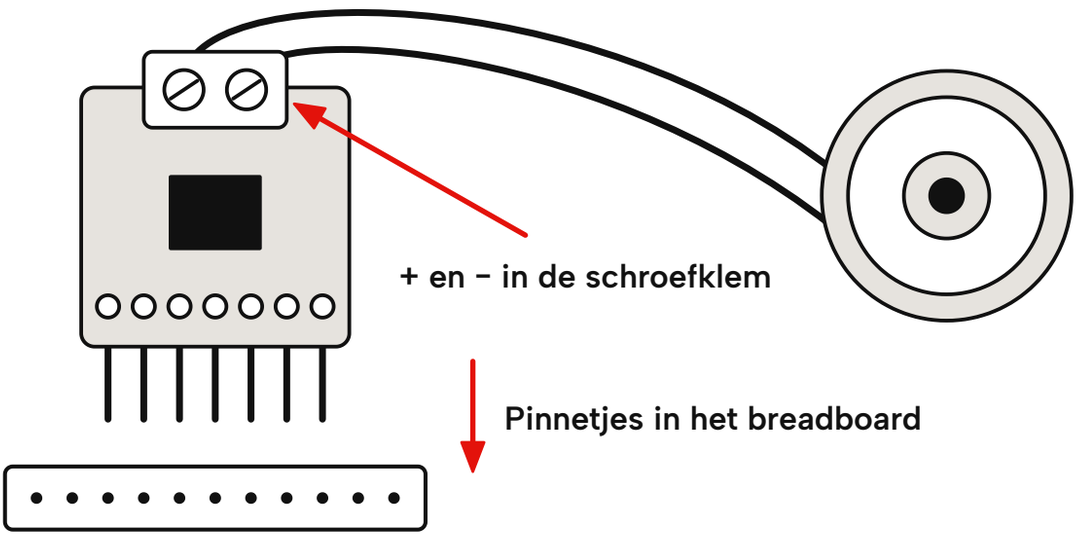
Naar de code →

- **Tip:** Koop het e-ink-scherm als module, dus met printplaatje en kabeltje. Bij een los scherm kun je de draden nergens in prikken.
- **Let op:** Gebruik een USB-kabel die ook data doorgeeft. Met een kabel die alleen oplaadt, ziet je laptop de NodeMCU niet.

1.2 Pinnetjes in het breadboard

De versterker heeft aan de onderkant een rij van 7 pinnetjes. Daarmee prik je hem zo in het breadboard. De knop gaat er ook in.

1. **Versterker plaatsen:** Druk de versterker met zijn pinnetjes recht in het breadboard, tot hij vastzit.
2. **Speaker vastzetten:** Draai de twee draden van de speaker vast in de schroefklem van de versterker, bij + en −.
3. **Knop plaatsen:** Prik de drukknop over de goot in het midden van het breadboard.



- **Let op:** De NodeMCU heeft ook pinnetjes, maar hij is net zo breed als het breadboard. Er blijft geen gaatje naast vrij, dus prik hem er niet in. De jumperdraden schuif je met het busje direct op zijn pinnetjes.
- **Let op:** De speaker gaat alleen op de versterker en nooit rechtstreeks op de NodeMCU.
- **Tip:** Koop de versterker met de pinnetjes er al op. Dan kun je hem meteen gebruiken.

STAP 2

2.1 Arduino IDE instellen

Nu de onderdelen klaarliggen, zet je de software op je laptop klaar. Dit hoef je maar één keer te doen.

Download Arduino IDE →

1. **Board toevoegen:** Ga naar **File → Preferences** (op een Mac: **Arduino IDE → Settings**) en plak bij *Additional boards manager URLs* deze link:
   `https://arduino.esp8266.com/stable/package_esp8266com_index.json`
2. **Board installeren:** Ga naar **Tools → Board → Boards Manager**, zoek op *esp8266* en installeer *esp8266 by ESP8266 Community* (versie 3 of hoger).
3. **Board kiezen:** Kies **Tools → Board → esp8266 → NodeMCU 1.0 (ESP-12E Module)**.
4. **Poort kiezen:** Sluit de NodeMCU aan met de USB-kabel en kies hem bij **Tools → Port**.

Tip: Zie je geen poort? Probeer een andere USB-kabel. Helpt dat niet, installeer dan de CH340-driver. Dat is het chipje dat op dit bord de USB regelt.

2.2 Libraries installeren

De code gebruikt twee libraries. Ga naar **Sketch → Include Library → Manage Libraries** en installeer ze allebei.

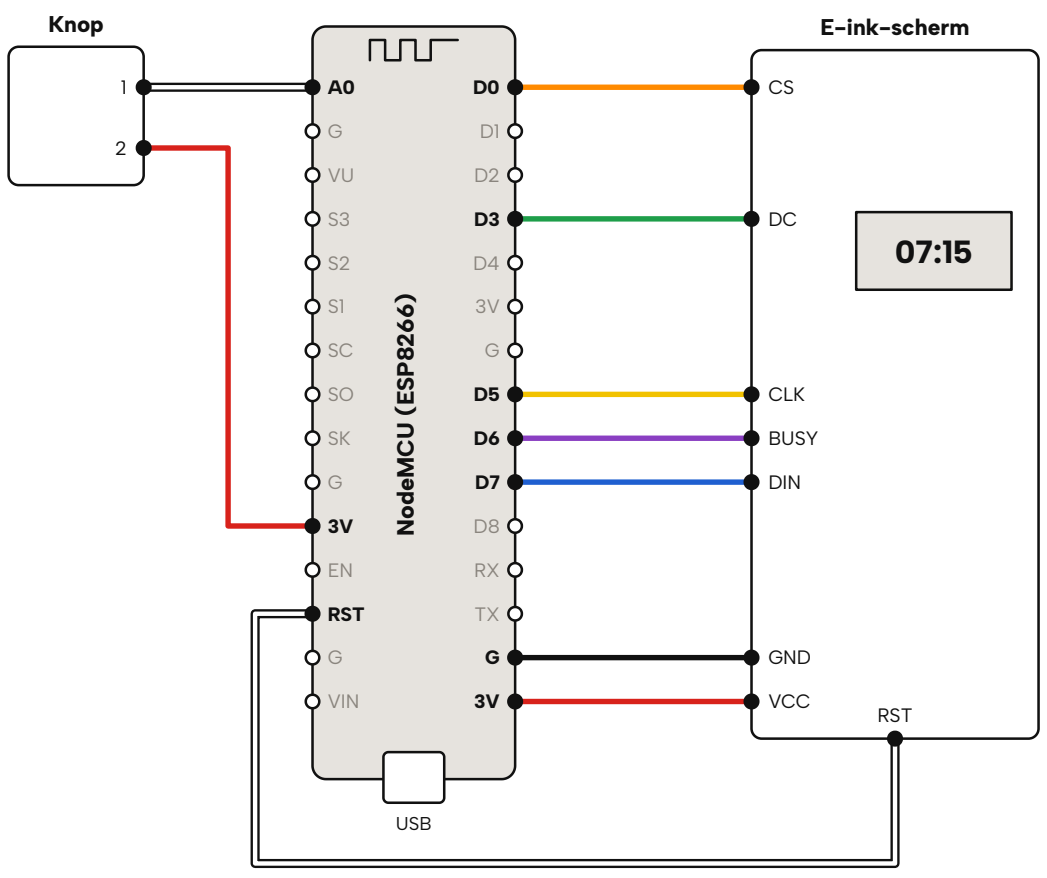
Library	Maker	Waarvoor
GxEPD2	Jean-Marc Zingg	Het e-ink-scherm. Kies bij de vraag *Install all*, dan komt Adafruit GFX mee.
ArduinoJson	Benoit Blanchon	De antwoorden van de API's lezen. Neem versie 7.

2.3 Bedrading aansluiten

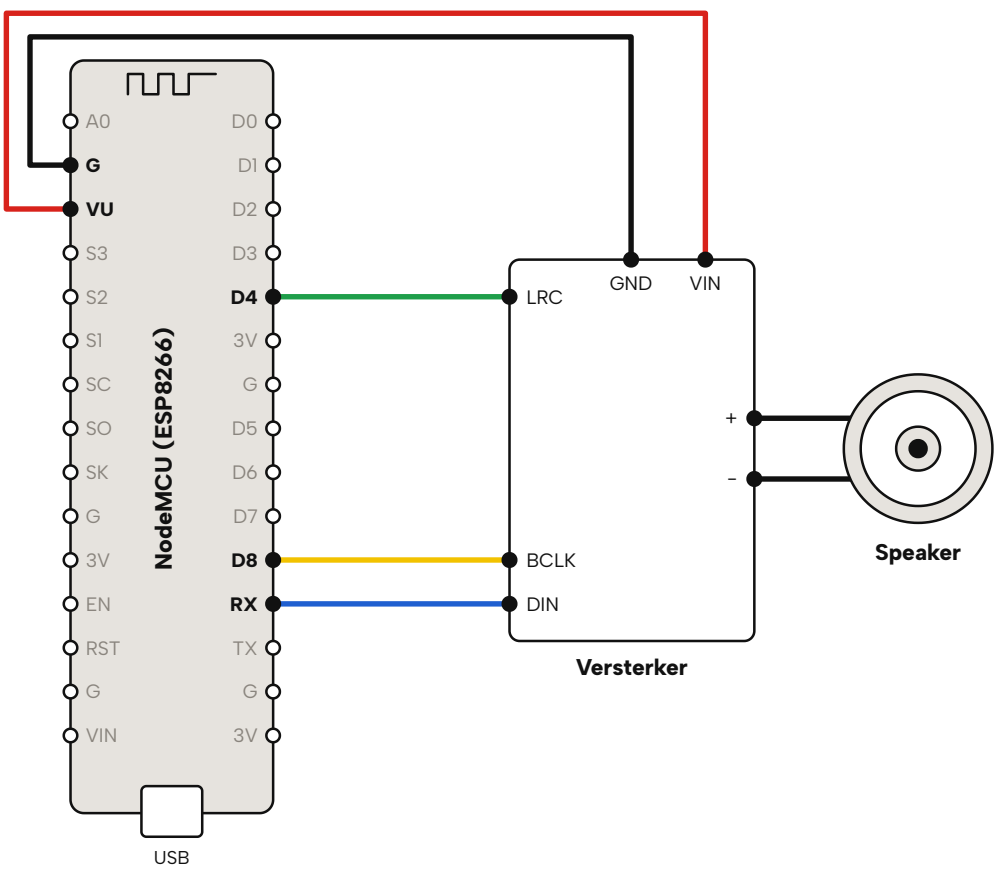
Verbind alles volgens de tabel. De namen in de laatste kolom staan zo op de NodeMCU. De kleuren zijn een voorstel, het gaat om de pin. Het bord heeft meerdere pinnen met 3V en met G. Welke je neemt maakt niet uit, in de schema's staat een handige keuze.

- **Let op:** Haal de USB-kabel eruit terwijl je draden aansluit.
- **Let op:** De versterker moet op D4, D8 en RX en het scherm op D5 en D7. Die pinnen liggen bij de ESP8266 vast.

Component	Draad	Pin op de NodeMCU
E-ink-scherm	VCC (rood)	3V
	GND (zwart)	G
	DIN (blauw)	D7 GPIO 13
	CLK (geel)	D5 GPIO 14
	CS (oranje)	D0 GPIO 16
	DC (groen)	D3 GPIO 0
	RST (wit)	RST
	BUSY (paars)	D6 GPIO 12
Versterker	VIN (rood)	VU 5 V van de USB
	GND (zwart)	G
	LRC (groen)	D4 GPIO 2
	BCLK (geel)	D8 GPIO 15
	DIN (blauw)	RX GPIO 3
Knop	pootje 1 (wit)	A0
	pootje 2 (rood)	3V



Het scherm en de knop. RST van het scherm gaat naar RST van het bord. De knop zit tussen A0 en 3V.



De versterker: VIN gaat naar VU, daar komt de 5 V van de USB uit.

Tip: Leg je de NodeMCU op zijn kop om bij de pinnetjes te kunnen? Dan zijn links en rechts omgedraaid. Kijk daarom steeds naar de naam die bij de pin staat.

STAP 3

3.1 Wat zijn de API's?

Een API is een adres op internet waar de wekker zelf gegevens kan ophalen. Deze wekker gebruikt er drie. Elke 10 minuten haalt hij ze opnieuw op en rekent hij uit wanneer je op moet. Wat je daarvoor nodig hebt, vul je in bovenaan de code, in het blok INSTELLINGEN.

API	Wat de wekker ermee doet	Wat je invult
Agenda	Leest je eerste afspraak van de dag.	De geheime link van je agenda
Trein (NS)	Zoekt de laatste trein waarmee je op tijd bent en ziet of die uitvalt.	Een sleutel en twee stations
Weer (Open-Meteo)	Kijkt of het regent als je onderweg bent.	De plek waar je woont

Open `SlimmeWekker.ino` in de Arduino IDE en vul eerst je wifi in, tussen de aanhalingstekens.

```
const char* WIFI_NAAM       = "JOUW_WIFI_NAAM";
const char* WIFI_WACHTWOORD = "JOUW_WIFI_WACHTWOORD";
```

3.2 De NS-sleutel aanvragen

NS geeft alleen treintijden aan wie een sleutel heeft. Die sleutel is gratis.

Naar het NS API-portaal →

1. **Account maken:** Klik op *Sign up*, vul je gegevens in en bevestig je e-mailadres.
2. **Abonneren:** Ga naar *Products*, kies *Ns-App*, geef je abonnement een naam en klik op *Subscribe*. Hier zit de Reisinformatie API in.
3. **Sleutel kopiëren:** Ga naar je *Profile*. Bij je abonnement staat de *Primary key*. Klik op *Show* en kopieer hem.
4. **Invullen:** Plak de sleutel in de code bij `NS_API_SLEUTEL`.

```
const char* NS_API_SLEUTEL = "JOUW_NS_API_SLEUTEL";
const char* NS_VAN         = "LEDN";   // stationscode vertrek: Leiden Centraal
const char* NS_NAAR        = "ASA";    // stationscode aankomst: Amsterdam Amstel
```

- **Let op:** De sleutel is van jou. Lever je de code in of deel je hem, zet dan weer `JOUW_NS_API_SLEUTEL` terug.
- **Goed om te weten:** Na het aanvragen kan het een paar minuten duren voordat de sleutel werkt.

3.3 Jouw traject

Bij `NS_VAN` vul je het station in waar je opstapt, bij `NS_NAAR` het station waar je uitstapt. NS gebruikt daarvoor korte codes.

Station	Code	Station	Code
Leiden Centraal	`LEDN`	**Utrecht Centraal**	`UT`
Amsterdam Centraal	`ASD`	**Rotterdam Centraal**	`RTD`
Amsterdam Amstel	`ASA`	**Den Haag Centraal**	`GVC`
Amsterdam Zuid	`ASDZ`	**Schiphol Airport**	`SHL`

Tip: Staat jouw station er niet bij? Zoek het station op Wikipedia. De code staat daar als afkorting.

3.4 De link van je agenda

De wekker leest je agenda via een geheime link. In Google Agenda vind je die zo.

1. **Instellingen openen:** Open Google Agenda op je laptop, klik rechtsboven op het tandwiel en kies *Instellingen*.
2. **Agenda kiezen:** Klik links onder *Instellingen voor mijn agenda's* op de agenda met je rooster.
3. **Link kopiëren:** Scrol naar *Agenda integreren* en kopieer het *Geheim adres in iCal-indeling*.
4. **Invullen:** Plak de link in de code bij `AGENDA_LINK`.

```
const char* AGENDA_LINK = "https://calendar.google.com/calendar/ical/JOUW_GEHEIME_LINK/basic.ics";
```

- **Tip:** Een rooster van school heeft vaak ook zo'n link. Die heet dan iCal, ICS of abonneren. Begint hij met `webcal://`, maak daar dan `https://` van.
- **Let op:** De wekker ziet alleen losse afspraken met een begintijd. Afspraken die zich herhalen en afspraken van een hele dag slaat hij over.
- **Let op:** Wie deze link heeft, kan je agenda lezen. Deel hem dus niet.

3.5 Het weer bij jou

Voor het weer is geen sleutel nodig. De wekker wil alleen weten waar je woont, in twee getallen.

1. **Plek opzoeken:** Zoek je straat op in Google Maps en klik met de rechtermuisknop op de kaart.
2. **Getallen kopiëren:** Bovenaan het menu staan twee getallen, bijvoorbeeld 52.16 en 4.49. Het eerste is de breedtegraad, het tweede de lengtegraad.
3. **Invullen:** Zet ze in de code, met een punt in plaats van een komma.

```
const float BREEDTEGRAAD = 52.16;
const float LENGTEGRAAD  = 4.49;
```

3.6 Jouw ochtend

Als laatste vertel je de wekker hoe lang jij 's ochtends nodig hebt. Daarmee rekent hij terug vanaf je eerste afspraak.

```
const int KLAARMAAK_MINUTEN          = 60;   // van wakker worden tot de deur uit
const int NAAR_STATION_MINUTEN       = 15;   // van huis naar het station
const int STATION_NAAR_SCHOOL_MINUTEN = 15;  // van het aankomststation tot je afspraak
const int MAX_EERDER_MINUTEN         = 30;   // zoveel eerder mag de wekker maximaal afgaan
const int VERVERS_MINUTEN            = 10;   // zo vaak worden de API's opnieuw opgehaald
```

Met deze getallen rekent de wekker zo:

Stap	Tijd	Hoe
Eerste afspraak	**09:00**	Uit je agenda.
Uiterlijk op het station	**08:45**	15 minuten eerder, voor het stuk van station naar school.
Trein	**07:57**	De laatste trein die voor 08:45 aankomt, hier om 08:33. Uit de NS-API.
Deur uit	**07:42**	15 minuten voor de trein.
Wekker	**06:42**	60 minuten om klaar te maken.

Valt de trein van 07:57 uit, dan neemt de wekker de trein ervoor en gaat hij eerder af. Dat doet hij maximaal 30 minuten eerder.

3.7 Uploaden en controleren

1. **Uploaden:** Klik op de pijl linksboven (Upload) en wacht tot er *Done uploading* staat.
2. **Meekijken:** Open **Tools → Serial Monitor** en zet hem rechtsonder op 115200 baud.
3. **Controleren:** Kijk of de drie API's antwoord geven. Dat ziet er zo uit.

```
=== Slimme wekker start op ===
Instellingen geladen: wekker aan, vaste wektijd 07:00, slim aan, geluid Piepjes
Scherm klaar.
Verbinden met wifi "JOUW_WIFI_NAAM"....
Wifi verbonden. Open in je browser: http://192.168.178.123  (of http://wekker.local)
Tijd opgehaald: 22:41, wo 7 okt.
API's ophalen om 22:41 (vrij geheugen: ... bytes)
Weer: 48 uren opgehaald.
Agenda: 212 afspraken gelezen.
Agenda: eerste afspraak 09:00 Werkcollege IoT
Trein: 6 reizen gekregen van NS.
Plan: wekker 06:42, deur uit 07:42, trein 07:57 spoor 5
```

Staat er iets anders? Dan zegt de melding wat er mis is.

Melding	Wat je doet
Trein: mislukt (HTTP 401)	Je NS-sleutel klopt niet of is nog niet actief. Kopieer hem opnieuw en wacht een paar minuten.
Trein: mislukt (HTTP 400 of 404)	Een stationscode klopt niet. Kijk bij stap 3.3.
Trein: geen trein gevonden	Er rijdt geen trein die op tijd aankomt. Kloppen de twee stations?
Agenda: mislukt	De link klopt niet. Kopieer hem opnieuw en kijk of hij met https:// begint.
Agenda: geen afspraak	Je agenda is leeg voor vandaag en morgen. De wekker gebruikt dan de vaste wektijd.
... is nog niet ingevuld	Er staat nog een invulplek met JOUW_ in de code.

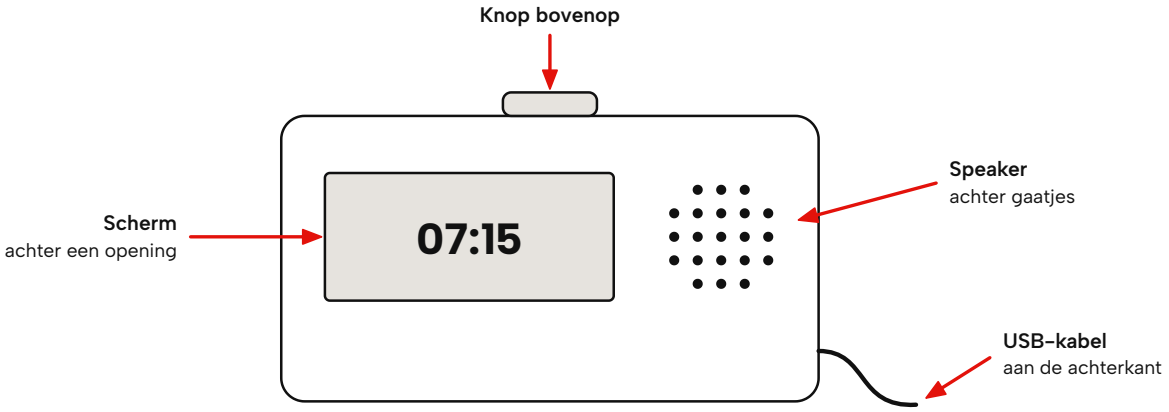
- **Tip:** Blijft het uploaden hangen op *Connecting...*? Houd dan de FLASH-knop op de NodeMCU ingedrukt tot het uploaden begint. Helpt dat niet, haal dan de draad van RX even los.
- **Tip:** Iets aangepast in je agenda? Op de webpagina van de wekker staat de knop API's ophalen. Dan hoef je geen 10 minuten te wachten.
- **Goed om te weten:** De eerste regel in de Serial Monitor bestaat uit rare tekens. Dat hoort zo.
- **Let op:** De ESP8266 werkt alleen op wifi van 2,4 GHz. Een hotspot van je telefoon kan ook.

STAP 4

4.1 De behuizing

Werkt alles los op tafel? Dan kan de wekker in een behuizing. Hoe die eruitziet mag je zelf weten: 3D-geprint, gelaserd of een bestaand doosje. Let wel op deze punten.

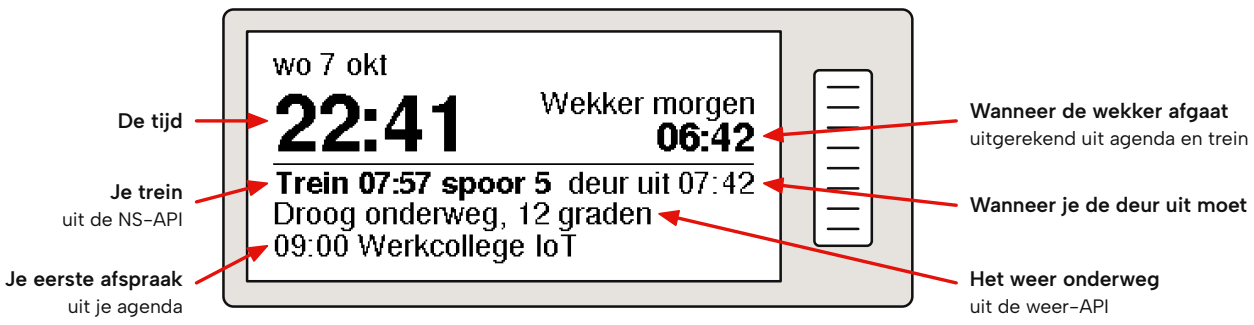
- **Scherm:** Maak aan de voorkant een opening voor het scherm. Het deel dat beeld geeft is ongeveer 67 × 29 mm. Meet je eigen scherm na.
- **Speaker:** Zet de speaker achter een paar gaatjes, anders klinkt het alarm dof.
- **Knop:** Bovenop, zodat je hem met je ogen dicht nog vindt.
- **Kabel:** Laat aan de achterkant een opening voor de USB-kabel.



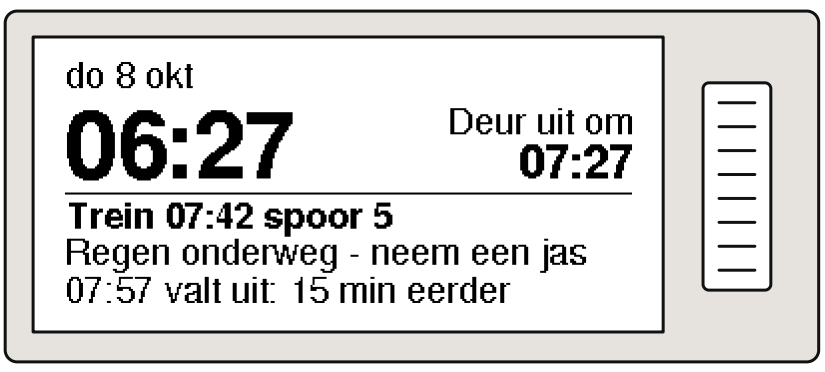
STAP 5

5.1 Wat staat er op het scherm?

Op het e-ink-scherm zie je in één blik wat de wekker uit de drie API's heeft gehaald.



Valt je trein uit, dan rekent de wekker opnieuw. Hij wekt je eerder en zet onderaan waarom.



Op een dag zonder afspraak, of als er geen internet is, gaat de wekker af op de vaste wektijd.

5.2 Vaste wektijd en geluid instellen

De vaste wektijd en het geluid stel je in op de webpagina van de wekker. Typ `http://wekker.local` in de browser van je telefoon of laptop. Die moet op dezelfde wifi zitten als de wekker.

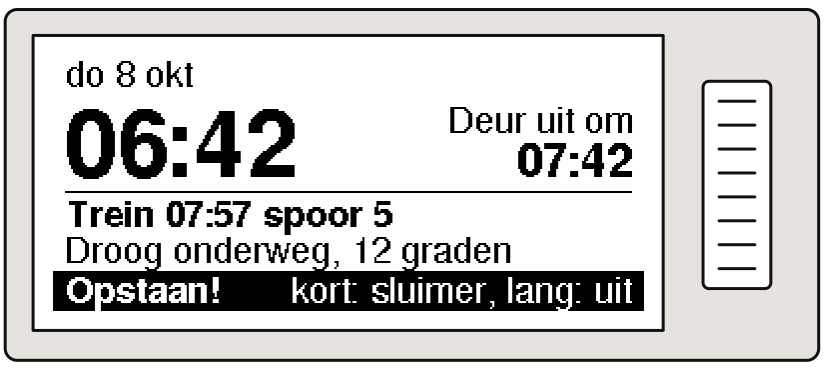
Tip: Werkt dat adres niet? Sluit de wekker aan op je laptop en open de Serial Monitor. Achter *Open in je browser* staat het adres met cijfers, bijvoorbeeld http://192.168.178.123.



Tip: Wil je het alarm testen? Zet het vinkje Slimme wektijd uit, zet de vaste wektijd 2 minuten vooruit en klik op Opslaan. Met Test geluid hoor je het geluid meteen, 5 seconden lang.

5.3 Alarm stoppen en sluimeren

Het alarm begint zacht en wordt langzaam harder. Met de knop bovenop zet je het uit.



Wat wil je?	Met de knop
Nog 9 minuten sluimeren	Kort drukken
Alarm uit	1 seconde vasthouden

Goed om te weten: Doe je niets, dan stopt het alarm vanzelf na 10 minuten. Daarna staat op het scherm wanneer je de deur uit moet.

API's erin, telefoon weg en klaar!

CODE

SlimmeWekker.ino

Maak in de Arduino IDE een nieuwe sketch met de naam *SlimmeWekker*, haal weg wat er al in staat en plak deze code erin. Vul daarna bovenaan bij INSTELLINGEN je wifi en de API's in, zoals in STAP 3.

```
/*
  Slimme wekker - NodeMCU (ESP8266) met e-ink-scherm en speaker
  Gemaakt door: Daan Ravensbergen (vak IoT)
*/

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <ESP8266mDNS.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>
#include <EEPROM.h>
#include <SPI.h>
#include <time.h>
#include <core_esp8266_i2s.h>
#include <ArduinoJson.h>
#include <GxEPD2_BW.h>
#include <Fonts/FreeSans9pt7b.h>
#include <Fonts/FreeSansBold9pt7b.h>
#include <Fonts/FreeSansBold12pt7b.h>
#include <Fonts/FreeSansBold24pt7b.h>

// ====================== 1. INSTELLINGEN ======================

// Wifi
const char* WIFI_NAAM       = "JOUW_WIFI_NAAM";
const char* WIFI_WACHTWOORD = "JOUW_WIFI_WACHTWOORD";

// API 1 - Trein (NS)
const char* NS_API_SLEUTEL = "JOUW_NS_API_SLEUTEL";
const char* NS_VAN         = "LEDN";   // stationscode vertrek: Leiden Centraal
const char* NS_NAAR        = "ASA";    // stationscode aankomst: Amsterdam Amstel

// API 2 - Weer (Open-Meteo)
const float BREEDTEGRAAD = 52.16;
const float LENGTEGRAAD  = 4.49;

// API 3 - Agenda (iCal)
const char* AGENDA_LINK = "https://calendar.google.com/calendar/ical/JOUW_GEHEIME_LINK/basic.ics";

// Jouw ochtend, in minuten
const int KLAARMAAK_MINUTEN          = 60;   // van wakker worden tot de deur uit
const int NAAR_STATION_MINUTEN       = 15;   // van huis naar het station
const int STATION_NAAR_SCHOOL_MINUTEN = 15;  // van het aankomststation tot je afspraak
const int MAX_EERDER_MINUTEN         = 30;   // zoveel eerder mag de wekker maximaal afgaan
const int VERVERS_MINUTEN            = 10;   // zo vaak worden de API's opnieuw opgehaald

// Naam op het netwerk (http://wekker.local)
const char* NAAM_OP_NETWERK = "wekker";

// Pinnen van het e-ink-scherm (SPI)
const int PIN_SCHERM_CS   = D0;   // GPIO 16
const int PIN_SCHERM_DC   = D3;   // GPIO 0
const int PIN_SCHERM_BUSY = D6;   // GPIO 12
const int PIN_SCHERM_CLK  = D5;   // GPIO 14
const int PIN_SCHERM_DIN  = D7;   // GPIO 13

// Pinnen van de versterker (I2S)
```

```
const int PIN_GELUID_LRC  = D4;   // GPIO 2
const int PIN_GELUID_BCLK = D8;   // GPIO 15
const int PIN_GELUID_DIN  = 3;    // GPIO 3, op het bord: RX

// Pin van de knop
const int PIN_KNOP = A0;

// Sluimeren en volume
const int SLUIMER_MINUTEN        = 9;    // zo lang duurt 1 keer sluimeren
const int VOLUME_START           = 15;   // volume waarmee het alarm begint (0-100)
const int VOLUME_MAX             = 70;   // hardste volume (0-100)
const int VOLUME_OPBOUW_SECONDEN = 60;   // in zoveel seconden van start naar max
const int ALARM_MAX_MINUTEN      = 10;   // daarna stopt het alarm vanzelf

// De vaste wektijd als er nog niets is opgeslagen
const int STANDAARD_WEK_UUR    = 7;
const int STANDAARD_WEK_MINUUT = 0;

// Scherm: na zoveel snelle verversingen volgt 1 volledige (haalt schaduwen weg)
const int VOLLEDIG_VERVERSEN_NA = 30;

// Type scherm: Waveshare 2,9 inch V2
#define SCHERM_TYPE GxEPD2_290_T94_V2

// Tijdzone Nederland, inclusief zomer- en wintertijd
const char* TIJDZONE     = "CET-1CEST,M3.5.0,M10.5.0/3";
const char* TIJDSERVER_1 = "pool.ntp.org";
const char* TIJDSERVER_2 = "time.nist.gov";

// Controle bij het compileren: deze pinnen liggen vast bij de ESP8266.
static_assert(PIN_SCHERM_CLK == 14 && PIN_SCHERM_DIN == 13, "Het scherm moet met CLK op D5 en DIN op D7.");
static_assert(PIN_GELUID_LRC == 2 && PIN_GELUID_BCLK == 15 && PIN_GELUID_DIN == 3,
              "De versterker moet op D4 (LRC), D8 (BCLK) en RX (DIN).");

// ====================== 2. DE ALARMGELUIDEN ======================

// Een geluid is een rijtje noten. Toon in Hz (0 = stilte), duur in milliseconden.
struct Noot {
  int toon;
  int duur;
};

const Noot NOTEN_PIEPJES[] = {
  {1760, 90}, {0, 90}, {1760, 90}, {0, 90}, {1760, 90}, {0, 90}, {1760, 90}, {0, 800}
};
const Noot NOTEN_RUSTIG[] = {
  {523, 350}, {659, 350}, {784, 350}, {1047, 700}, {0, 1000}
};
const Noot NOTEN_VROLIJK[] = {
  {659, 180}, {784, 180}, {880, 180}, {784, 180}, {659, 180}, {523, 180}, {587, 180}, {659, 360}, {0, 900}
};

struct Geluid {
  const char* naam;
  const Noot* noten;
  int         aantal;
};

const Geluid GELUIDEN[] = {
  {"Piepjes", NOTEN_PIEPJES, sizeof(NOTEN_PIEPJES) / sizeof(Noot)},
  {"Rustig",  NOTEN_RUSTIG,  sizeof(NOTEN_RUSTIG)  / sizeof(Noot)},
  {"Vrolijk", NOTEN_VROLIJK, sizeof(NOTEN_VROLIJK) / sizeof(Noot)}
};
const int AANTAL_GELUIDEN = sizeof(GELUIDEN) / sizeof(Geluid);

// ====================== 3. WAT DE WEKKER BIJHOUDT ======================

enum Toestand { RUST, ALARM, SLUIMER };                         // wat de wekker nu doet
enum KnopActie { KNOP_NIETS, KNOP_KORT, KNOP_LANG };

struct Afspraak {
```

```
  bool   gevonden;
  time_t begin;
  char   titel[40];
};

struct Reis {
  bool   gevonden;
  time_t vertrek;          // werkelijke vertrektijd
  time_t vertrekGepland;   // geplande vertrektijd
  time_t aankomst;
  bool   valtUit;
  char   spoor[8];
};

// Het plan voor de ochtend: wat de wekker uit de 3 API's heeft uitgerekend.
struct Plan {
  bool     geldig;         // is er een afspraak met een bruikbare trein?
  Afspraak afspraak;
  Reis     trein;          // de trein die je neemt
  time_t   wekTijd;
  time_t   deurUit;
  int      eerderMinuten;  // zoveel minuten eerder dan normaal
  char     reden[40];      // uitleg voor op het scherm
};

// Zo staan de instellingen in het geheugen (EEPROM) dat de stroom overleeft.
struct Bewaard {
  uint16_t kenmerk;   // vast getal: zo weten we of er al iets is opgeslagen
  uint8_t  uur;
  uint8_t  minuut;
  uint8_t  geluid;
  uint8_t  aan;
  uint8_t  slim;
};
const uint16_t BEWAARD_KENMERK = 0x5A18;

// De helft van het scherm per keer tekenen spaart geheugen voor de beveiligde verbinding met de API's.
GxEPD2_BW<SCHERM_TYPE, SCHERM_TYPE::HEIGHT / 2>
  scherm(SCHERM_TYPE(PIN_SCHERM_CS, PIN_SCHERM_DC, -1, PIN_SCHERM_BUSY));

ESP8266WebServer server(80);

// Instellingen die je op de webpagina kiest (worden bewaard)
int  wekUur        = STANDAARD_WEK_UUR;
int  wekMinuut     = STANDAARD_WEK_MINUUT;
int  gekozenGeluid = 0;
bool wekkerAan     = true;
bool slimAan       = true;   // wektijd uitrekenen met agenda en trein

// Gegevens van de API's
Plan   plan;
time_t laatstBijgewerkt = 0;         // laatste keer dat agenda en trein goed zijn opgehaald

const int WEER_MAX = 48;             // 2 dagen, 1 regel per uur
time_t weerTijd[WEER_MAX];
int8_t weerGraden[WEER_MAX];
bool   weerRegen[WEER_MAX];
int    weerAantal = 0;

unsigned long laatsteVerversPoging = 0;
bool          verversNu            = true;    // true = zo snel mogelijk de API's ophalen

// Toestand van het alarm
Toestand      toestand          = RUST;
unsigned long alarmGestartOp    = 0;    // millis() toen het alarm begon
time_t        sluimerTot        = 0;    // tijdstip waarop het alarm terugkomt
int           laatsteAlarmDag   = -1;   // dag van het jaar waarop het alarm al is afgegaan

// Geluid (zie deel 6)
bool          geluidSpeelt   = false;
int           geluidKeuze    = 0;
```

```
int           geluidVolume   = 0;       // 0-100
bool          testBezig      = false;
unsigned long testGestartOp  = 0;

// Scherm
bool schermVerversen  = true;   // true = het scherm moet opnieuw getekend worden
int  deelVerversingen = 0;
bool eersteTekening   = true;
int  vorigeMinuut     = -1;

// Overig
bool          wifiWasVerbonden     = false;
bool          tijdWasBekend        = false;
unsigned long laatsteWifiPoging    = 0;
KnopActie     knopActieKlaar       = KNOP_NIETS;   // druk op de knop die nog afgehandeld moet worden

const int SAMPLES_PER_SECONDE = 11025;
const int KNOP_LANG_MS        = 1000;
const int KNOP_DREMPEL        = 512;    // A0 geeft 0 (los) tot 1023 (ingedrukt)
const int TEST_DUUR_MS        = 5000;

const char* DAGEN[]   = {"zo", "ma", "di", "wo", "do", "vr", "za"};
const char* MAANDEN[] = {"jan", "feb", "mrt", "apr", "mei", "jun", "jul", "aug", "sep", "okt", "nov", "dec"};

// ====================== 4. TIJD ======================

// Is de tijd al opgehaald? Zonder wifi staat de klok van de ESP8266 op 1970.
bool tijdBekend() {
  return time(nullptr) > 1700000000;
}

// Maakt van uur en minuut een tekst zoals "07:05".
String tijdTekst(int uur, int minuut) {
  char tekst[8];
  snprintf(tekst, sizeof(tekst), "%02d:%02d", uur, minuut);
  return String(tekst);
}

// Hetzelfde, maar voor een tijdstip in seconden sinds 1970.
String klokTekst(time_t moment) {
  struct tm t;
  localtime_r(&moment, &t);
  return tijdTekst(t.tm_hour, t.tm_min);
}

// Aantal dagen sinds 1 januari 1970 voor een datum.
long dagenSinds1970(int jaar, int maand, int dag) {
  jaar -= (maand <= 2) ? 1 : 0;
  long tijdperk = (jaar >= 0 ? jaar : jaar - 399) / 400;
  long jaarIn   = jaar - tijdperk * 400;
  long dagIn    = (153 * (maand + (maand > 2 ? -3 : 9)) + 2) / 5 + dag - 1;
  long dagen    = jaarIn * 365 + jaarIn / 4 - jaarIn / 100 + dagIn;
  return tijdperk * 146097L + dagen - 719468L;
}

// Datum en tijd in UTC naar seconden sinds 1970.
time_t utcNaarMoment(int jaar, int maand, int dag, int uur, int minuut, int seconde) {
  return (time_t)dagenSinds1970(jaar, maand, dag) * 86400 + uur * 3600L + minuut * 60 + seconde;
}

// Datum en tijd in Nederlandse tijd naar seconden sinds 1970.
time_t lokaalNaarMoment(int jaar, int maand, int dag, int uur, int minuut, int seconde) {
  struct tm t = {};
  t.tm_year = jaar - 1900; t.tm_mon = maand - 1; t.tm_mday = dag;
  t.tm_hour = uur; t.tm_min = minuut; t.tm_sec = seconde; t.tm_isdst = -1;
  return mktime(&t);
}

// Verschil tussen Nederlandse tijd en UTC in seconden op dat moment (zomer: 7200).
long utcVerschil(time_t moment) {
  struct tm t;
```

```
  localtime_r(&moment, &t);
  return (long)(utcNaarMoment(t.tm_year + 1900, t.tm_mon + 1, t.tm_mday, t.tm_hour, t.tm_min, t.tm_sec) - moment);
}

// NS-tijd zoals "2026-10-08T07:42:00+0200" naar seconden sinds 1970. 0 = mislukt.
time_t leesNsTijd(const char* tekst) {
  if (!tekst) return 0;
  int jaar, maand, dag, uur, minuut, seconde;
  if (sscanf(tekst, "%4d-%2d-%2dT%2d:%2d:%2d", &jaar, &maand, &dag, &uur, &minuut, &seconde) != 6) return 0;
  long verschil = 0;
  if (strlen(tekst) >= 22 && (tekst[19] == '+' || tekst[19] == '-')) {
    const char* p = tekst + 20;
    int u = (p[0] - '0') * 10 + (p[1] - '0');
    int m = 0;
    p += 2;
    if (*p == ':') p++;
    if (p[0] && p[1]) m = (p[0] - '0') * 10 + (p[1] - '0');
    verschil = u * 3600L + m * 60L;
    if (tekst[19] == '-') verschil = -verschil;
  }
  return utcNaarMoment(jaar, maand, dag, uur, minuut, seconde) - verschil;
}

// Agendaregel zoals "DTSTART:20261008T070000Z" of "DTSTART;TZID=Europe/Amsterdam:20261008T090000".
// Geeft false bij een afspraak die de hele dag duurt.
bool leesAgendaTijd(const char* regel, time_t &moment) {
  const char* dubbelePunt = strchr(regel, ':');
  if (!dubbelePunt) return false;
  const char* w = dubbelePunt + 1;
  if (strlen(w) < 15 || w[8] != 'T') return false;
  int jaar, maand, dag, uur, minuut, seconde;
  if (sscanf(w, "%4d%2d%2dT%2d%2d%2d", &jaar, &maand, &dag, &uur, &minuut, &seconde) != 6) return false;
  moment = (w[15] == 'Z') ? utcNaarMoment(jaar, maand, dag, uur, minuut, seconde)
                          : lokaalNaarMoment(jaar, maand, dag, uur, minuut, seconde);
  return true;
}

// Begin van de dag (00:00 Nederlandse tijd). dagVerder 0 = vandaag, 1 = morgen.
time_t beginVanDag(time_t nu, int dagVerder) {
  struct tm t;
  localtime_r(&nu, &t);
  t.tm_hour = 0; t.tm_min = 0; t.tm_sec = 0;
  t.tm_mday += dagVerder;
  t.tm_isdst = -1;
  return mktime(&t);
}

bool zelfdeDag(time_t a, time_t b) {
  struct tm ta, tb;
  localtime_r(&a, &ta);
  localtime_r(&b, &tb);
  return ta.tm_yday == tb.tm_yday && ta.tm_year == tb.tm_year;
}

// De vaste wektijd op de dag van 'nu' (dagVerder 1 = een dag later).
time_t vasteWektijdOp(time_t nu, int dagVerder) {
  return beginVanDag(nu, dagVerder) + wekUur * 3600L + wekMinuut * 60L;
}

// Is er een plan uit de API's dat nog over de toekomst gaat?
bool planBruikbaar(time_t nu) {
  return slimAan && plan.geldig && nu < plan.deurUit;
}

// Het moment waarop de wekker de volgende keer afgaat.
//   - Dag met een afspraak: de uitgerekende wektijd uit het plan.
//   - Dag zonder afspraak, slim uit of geen gegevens: de vaste wektijd.
time_t volgendeWekMoment(time_t nu) {
  struct tm t;
  localtime_r(&nu, &t);
  bool   vandaagGewekt = (laatsteAlarmDag == t.tm_yday);
```

```
  time_t vast          = vasteWektijdOp(nu, 0);

  if (planBruikbaar(nu)) {
    if (!zelfdeDag(plan.wekTijd, nu)) {
      // Het plan is voor morgen. Vandaag geldt nog de vaste wektijd, als die niet voorbij is.
      if (!vandaagGewekt && nu < vast + 120) return vast;
      return plan.wekTijd;
    }
    if (!vandaagGewekt) return plan.wekTijd;
  } else if (!vandaagGewekt && nu < vast + 120) {
    return vast;
  }
  return vasteWektijdOp(nu, 1);
}

// ====================== 5. INSTELLINGEN BEWAREN ======================

void instellingenLaden() {
  EEPROM.begin(16);
  Bewaard b;
  EEPROM.get(0, b);
  if (b.kenmerk == BEWAARD_KENMERK && b.uur <= 23 && b.minuut <= 59 && b.geluid < AANTAL_GELUIDEN) {
    wekUur        = b.uur;
    wekMinuut     = b.minuut;
    gekozenGeluid = b.geluid;
    wekkerAan     = b.aan != 0;
    slimAan       = b.slim != 0;
  } else {
    Serial.println("Nog geen instellingen bewaard, de wekker begint met de standaard wektijd.");
  }
  Serial.printf("Instellingen geladen: wekker %s, vaste wektijd %s, slim %s, geluid %s\n",
                wekkerAan ? "aan" : "uit", tijdTekst(wekUur, wekMinuut).c_str(),
                slimAan ? "aan" : "uit", GELUIDEN[gekozenGeluid].naam);
}

void instellingenBewaren() {
  Bewaard b;
  b.kenmerk = BEWAARD_KENMERK;
  b.uur     = wekUur;
  b.minuut  = wekMinuut;
  b.geluid  = gekozenGeluid;
  b.aan     = wekkerAan ? 1 : 0;
  b.slim    = slimAan ? 1 : 0;
  EEPROM.put(0, b);
  bool gelukt = EEPROM.commit();
  Serial.printf("Instellingen %s: wekker %s, vaste wektijd %s, slim %s, geluid %s\n",
                gelukt ? "bewaard" : "NIET bewaard", wekkerAan ? "aan" : "uit",
                tijdTekst(wekUur, wekMinuut).c_str(), slimAan ? "aan" : "uit", GELUIDEN[gekozenGeluid].naam);
}

// ====================== 6. GELUID (I2S) ======================

// De wekker maakt zelf de tonen: een sinusgolf per noot, dus zonder geluidsbestanden.
// De versterker heeft een buffer voor ongeveer 40 milliseconden geluid. geluidVoeden()
// vult die bij en moet daarom heel vaak worden aangeroepen: in loop() en terwijl het
// scherm ververst (zie schermWacht in deel 12).

int16_t  SINUS[256];          // 1 golf van een sinus, uitgerekend bij het opstarten
int      nootNummer   = 0;    // welke noot van het geluid nu speelt
long     sampleInNoot = 0;    // hoe ver we in die noot zijn
uint32_t fase         = 0;    // plek in de golf (de bovenste 8 bits wijzen in SINUS)
uint32_t faseStap     = 0;    // zoveel schuift de plek op per sample: hoe groter, hoe hoger de toon

void geluidKlaarzetten() {
  for (int i = 0; i < 256; i++) SINUS[i] = (int16_t)(sin(i * 2.0 * PI / 256.0) * 32767.0);
}

// Zet de toon van de huidige noot klaar.
void nootStarten() {
  int toon = GELUIDEN[geluidKeuze].noten[nootNummer].toon;
  sampleInNoot = 0;
```

```
  fase         = 0;
  faseStap     = (uint32_t)(((uint64_t)toon << 32) / SAMPLES_PER_SECONDE);
}

void geluidStarten(int keuze, int volume) {
  if (keuze < 0 || keuze >= AANTAL_GELUIDEN) keuze = 0;
  if (!geluidSpeelt) {
    i2s_begin();                          // neemt D4, D8 en RX in gebruik
    i2s_set_rate(SAMPLES_PER_SECONDE);
  }
  geluidKeuze  = keuze;
  geluidVolume = volume;
  nootNummer   = 0;
  nootStarten();
  geluidSpeelt = true;
}

void geluidStoppen() {
  if (!geluidSpeelt) return;
  geluidSpeelt = false;
  i2s_end();                              // geeft de pinnen weer vrij, de versterker valt stil
}

// Vult de buffer van de versterker bij tot hij vol is.
void geluidVoeden() {
  if (!geluidSpeelt) return;
  const Geluid& geluid = GELUIDEN[geluidKeuze];

  // Het volume in het kwadraat klinkt voor je oor als een gelijkmatige opbouw.
  long sterkte = (long)geluidVolume * geluidVolume * 3;      // 0 tot 30000
  long rand    = SAMPLES_PER_SECONDE * 5 / 1000;             // 5 ms zacht begin en eind per noot

  while (!i2s_is_full()) {
    const Noot& noot     = geluid.noten[nootNummer];
    long        nootDuur = (long)noot.duur * SAMPLES_PER_SECONDE / 1000;
    long        waarde   = 0;

    if (noot.toon > 0) {
      fase  += faseStap;
      waarde = ((long)SINUS[fase >> 24] * sterkte) >> 15;
      // Zacht begin en eind van de noot. Dat voorkomt getik.
      if (sampleInNoot < rand)                 waarde = waarde * sampleInNoot / rand;
      else if (nootDuur - sampleInNoot < rand) waarde = waarde * (nootDuur - sampleInNoot) / rand;
    }

    uint16_t sample = (uint16_t)(int16_t)waarde;
    i2s_write_sample_nb(((uint32_t)sample << 16) | sample);   // links en rechts hetzelfde

    sampleInNoot++;
    if (sampleInNoot >= nootDuur) {            // volgende noot
      nootNummer = (nootNummer + 1) % geluid.aantal;
      nootStarten();
    }
  }
}

// ====================== 7. INTERNET: GEGEVENS OPHALEN ======================

// Zijn de invulplekken bovenaan de code al vervangen?
bool nsIngevuld() {
  return strstr(NS_API_SLEUTEL, "JOUW_") == nullptr && strlen(NS_API_SLEUTEL) > 0;
}

bool agendaIngevuld() {
  return strstr(AGENDA_LINK, "JOUW_") == nullptr && strncmp(AGENDA_LINK, "https://", 8) == 0;
}

// Opent een beveiligde verbinding (https). HTTP 1.0 zorgt dat het antwoord in een keer leesbaar is.
bool httpOpenen(HTTPClient &http, BearSSL::WiFiClientSecure &client, const String &url) {
  client.setInsecure();   // prototype: het certificaat van de server wordt niet gecontroleerd
  http.useHTTP10(true);
```

```
  http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
  http.setTimeout(15000);
  return http.begin(client, url);
}

// Leest een regel uit het antwoord. Te lange regels worden afgekapt.
bool leesRegel(HTTPClient &http, WiFiClient &stroom, char* regel, size_t grootte) {
  size_t n = 0;
  unsigned long laatste = millis();
  while (true) {
    if (stroom.available()) {
      char c = stroom.read();
      laatste = millis();
      if (c == '\n') { regel[n] = 0; return true; }
      if (c != '\r' && n < grootte - 1) regel[n++] = c;
    } else if (!http.connected() || millis() - laatste > 5000) {
      regel[n] = 0;
      return n > 0;
    } else {
      delay(1);
    }
  }
}

// ====================== 8. API 3: AGENDA (iCal) ======================

// Zoekt de eerste afspraak van vandaag. Is die al begonnen, dan de eerste van morgen.
// Let op: afspraken die zich herhalen (RRULE) en afspraken van een hele dag worden overgeslagen.
bool agendaOphalen(time_t nu, Afspraak &resultaat) {
  BearSSL::WiFiClientSecure client;
  HTTPClient http;
  if (!httpOpenen(http, client, AGENDA_LINK)) return false;
  int code = http.GET();
  if (code != 200) {
    Serial.printf("Agenda: mislukt (HTTP %d). Klopt de link bij AGENDA_LINK?\n", code);
    http.end();
    return false;
  }

  time_t vandaag   = beginVanDag(nu, 0);
  time_t morgen    = beginVanDag(nu, 1);
  time_t overmorgen = beginVanDag(nu, 2);
  Afspraak eersteVandaag = {false, 0, ""};
  Afspraak eersteMorgen  = {false, 0, ""};

  WiFiClient &stroom = http.getStream();
  char   regel[160];
  bool   inAfspraak = false, heeftBegin = false, herhaalt = false;
  time_t begin = 0;
  char   titel[40] = "";
  int    overgeslagen = 0, geteld = 0;

  while (leesRegel(http, stroom, regel, sizeof(regel))) {
    if (strcmp(regel, "BEGIN:VEVENT") == 0) {
      inAfspraak = true; heeftBegin = false; herhaalt = false; titel[0] = 0;
    } else if (!inAfspraak) {
      continue;
    } else if (strncmp(regel, "DTSTART", 7) == 0) {
      heeftBegin = leesAgendaTijd(regel, begin);
    } else if (strncmp(regel, "SUMMARY", 7) == 0) {
      const char* dubbelePunt = strchr(regel, ':');
      if (dubbelePunt) {
        // Titel overnemen zonder de schuine strepen die iCal voor komma's zet.
        size_t n = 0;
        for (const char* p = dubbelePunt + 1; *p && n < sizeof(titel) - 1; p++) {
          if (*p != '\\') titel[n++] = *p;
        }
        titel[n] = 0;
      }
    } else if (strncmp(regel, "RRULE", 5) == 0) {
      herhaalt = true;
```

```
    } else if (strcmp(regel, "END:VEVENT") == 0) {
      inAfspraak = false;
      geteld++;
      if (herhaalt) { overgeslagen++; continue; }
      if (!heeftBegin) continue;
      Afspraak* plek = nullptr;
      if (begin >= vandaag && begin < morgen)         plek = &eersteVandaag;
      else if (begin >= morgen && begin < overmorgen) plek = &eersteMorgen;
      if (plek && (!plek->gevonden || begin < plek->begin)) {
        plek->gevonden = true;
        plek->begin    = begin;
        strncpy(plek->titel, titel, sizeof(plek->titel) - 1);
        plek->titel[sizeof(plek->titel) - 1] = 0;
      }
    }
  }
  http.end();
  Serial.printf("Agenda: %d afspraken gelezen", geteld);
  if (overgeslagen > 0) Serial.printf(", %d herhalende overgeslagen", overgeslagen);
  Serial.println(".");

  if (eersteVandaag.gevonden && nu < eersteVandaag.begin) resultaat = eersteVandaag;
  else resultaat = eersteMorgen;
  return true;
}

// ====================== 9. API 1: TREIN (NS) ======================

// Zoekt treinen die voor 'uiterlijk' aankomen.
//   gepland = de trein die je normaal zou nemen (ook als hij uitvalt)
//   gekozen = de laatste trein die echt rijdt en op tijd aankomt
bool treinOphalen(time_t uiterlijk, Reis &gepland, Reis &gekozen) {
  gepland.gevonden = false;
  gekozen.gevonden = false;

  // De aankomsttijd in de vorm die NS wil: 2026-10-08T08:45:00+02:00 (met %3A voor : en %2B voor +)
  struct tm t;
  localtime_r(&uiterlijk, &t);
  long verschil = utcVerschil(uiterlijk);
  char wanneer[96];
  snprintf(wanneer, sizeof(wanneer), "%04d-%02d-%02dT%02d%%3A%02d%%3A00%s%02ld%%3A%02ld",
           t.tm_year + 1900, t.tm_mon + 1, t.tm_mday, t.tm_hour, t.tm_min,
           verschil >= 0 ? "%2B" : "-", labs(verschil) / 3600, (labs(verschil) % 3600) / 60);

  String url = "https://gateway.apiportal.ns.nl/reisinformatie-api/api/v3/trips?fromStation=";
  url += NS_VAN; url += "&toStation="; url += NS_NAAR;
  url += "&dateTime="; url += wanneer; url += "&searchForArrival=true";

  BearSSL::WiFiClientSecure client;
  HTTPClient http;
  if (!httpOpenen(http, client, url)) return false;
  http.addHeader("Ocp-Apim-Subscription-Key", NS_API_SLEUTEL);
  int code = http.GET();
  if (code != 200) {
    Serial.printf("Trein: mislukt (HTTP %d).", code);
    if (code == 401 || code == 403) Serial.print(" Je NS-sleutel klopt niet of is nog niet actief.");
    if (code == 400 || code == 404) Serial.print(" Kloppen de stationscodes bij NS_VAN en NS_NAAR?");
    Serial.println();
    http.end();
    return false;
  }

  // Het antwoord van NS is groot. Het filter bewaart alleen de velden die we gebruiken.
  JsonDocument filter;
  filter["trips"][0]["status"] = true;
  JsonObject deel = filter["trips"][0]["legs"][0].to<JsonObject>();
  deel["cancelled"] = true;
  deel["origin"]["plannedDateTime"] = true;
  deel["origin"]["actualDateTime"] = true;
  deel["origin"]["plannedTrack"] = true;
  deel["origin"]["actualTrack"] = true;
```

```
  deel["destination"]["plannedDateTime"] = true;
  deel["destination"]["actualDateTime"] = true;

  JsonDocument doc;
  DeserializationError fout = deserializeJson(doc, http.getStream(),
                                              DeserializationOption::Filter(filter),
                                              DeserializationOption::NestingLimit(20));
  http.end();
  if (fout) {
    Serial.printf("Trein: het antwoord van NS is niet te lezen (%s).\n", fout.c_str());
    return false;
  }

  int aantal = 0;
  for (JsonObject reis : doc["trips"].as<JsonArray>()) {
    JsonArray delen = reis["legs"];
    if (delen.size() == 0) continue;
    aantal++;
    JsonObject vertrekPunt  = delen[0]["origin"];
    JsonObject aankomstPunt = delen[delen.size() - 1]["destination"];

    const char* vertrekGeplandTekst  = vertrekPunt["plannedDateTime"];
    const char* vertrekEchtTekst     = vertrekPunt["actualDateTime"] | vertrekGeplandTekst;
    const char* aankomstGeplandTekst = aankomstPunt["plannedDateTime"];
    const char* aankomstEchtTekst    = aankomstPunt["actualDateTime"] | aankomstGeplandTekst;
    time_t vertrekGepland  = leesNsTijd(vertrekGeplandTekst);
    time_t vertrekEcht     = leesNsTijd(vertrekEchtTekst);
    time_t aankomstGepland = leesNsTijd(aankomstGeplandTekst);
    time_t aankomstEcht    = leesNsTijd(aankomstEchtTekst);
    if (vertrekGepland == 0 || aankomstGepland == 0) continue;

    bool valtUit = strcmp(reis["status"] | "", "CANCELLED") == 0;
    for (JsonObject d : delen) {
      if (d["cancelled"] | false) valtUit = true;
    }
    const char* spoor = vertrekPunt["actualTrack"] | (vertrekPunt["plannedTrack"] | "?");

    if (aankomstGepland <= uiterlijk && (!gepland.gevonden || vertrekGepland > gepland.vertrekGepland)) {
      gepland.gevonden = true;
      gepland.vertrek = vertrekEcht; gepland.vertrekGepland = vertrekGepland; gepland.aankomst = aankomstGepland;
      gepland.valtUit = valtUit;
      strncpy(gepland.spoor, spoor, sizeof(gepland.spoor) - 1);
      gepland.spoor[sizeof(gepland.spoor) - 1] = 0;
    }
    if (!valtUit && aankomstEcht <= uiterlijk && (!gekozen.gevonden || vertrekEcht > gekozen.vertrek)) {
      gekozen.gevonden = true;
      gekozen.vertrek = vertrekEcht; gekozen.vertrekGepland = vertrekGepland; gekozen.aankomst = aankomstEcht;
      gekozen.valtUit = false;
      strncpy(gekozen.spoor, spoor, sizeof(gekozen.spoor) - 1);
      gekozen.spoor[sizeof(gekozen.spoor) - 1] = 0;
    }
  }
  Serial.printf("Trein: %d reizen gekregen van NS.\n", aantal);
  return true;
}

// ====================== 10. API 2: WEER (Open-Meteo) ======================

bool weerOphalen() {
  char url[220];
  snprintf(url, sizeof(url),
           "https://api.open-meteo.com/v1/forecast?latitude=%.4f&longitude=%.4f"
           "&hourly=temperature_2m,precipitation_probability,precipitation"
           "&forecast_days=2&timeformat=unixtime",
           BREEDTEGRAAD, LENGTEGRAAD);

  BearSSL::WiFiClientSecure client;
  HTTPClient http;
  if (!httpOpenen(http, client, url)) return false;
  int code = http.GET();
  if (code != 200) {
```

```
    Serial.printf("Weer: mislukt (HTTP %d).\n", code);
    http.end();
    return false;
  }
  JsonDocument filter;
  filter["hourly"]["time"] = true;
  filter["hourly"]["temperature_2m"] = true;
  filter["hourly"]["precipitation_probability"] = true;
  filter["hourly"]["precipitation"] = true;

  JsonDocument doc;
  DeserializationError fout = deserializeJson(doc, http.getStream(), DeserializationOption::Filter(filter));
  http.end();
  if (fout) {
    Serial.printf("Weer: het antwoord is niet te lezen (%s).\n", fout.c_str());
    return false;
  }

  JsonArray tijden  = doc["hourly"]["time"];
  JsonArray graden  = doc["hourly"]["temperature_2m"];
  JsonArray kansen  = doc["hourly"]["precipitation_probability"];
  JsonArray regens  = doc["hourly"]["precipitation"];
  int n = min((int)tijden.size(), WEER_MAX);
  for (int i = 0; i < n; i++) {
    float g = graden[i] | 0.0f;
    weerTijd[i]   = (time_t)(tijden[i] | 0L);
    weerGraden[i] = (int8_t)(g >= 0 ? g + 0.5f : g - 0.5f);
    // Regen: minstens 0,2 mm in dat uur, of een kans van 60% of meer.
    weerRegen[i]  = (regens[i] | 0.0f) >= 0.2f || (kansen[i] | 0) >= 60;
  }
  weerAantal = n;
  Serial.printf("Weer: %d uren opgehaald.\n", n);
  return n > 0;
}

// Maakt de weerregel voor de periode tussen 'van' en 'tot'.
String weerTekst(time_t van, time_t tot, bool onderweg) {
  bool regen = false, gradenBekend = false;
  int  graden = 0;
  for (int i = 0; i < weerAantal; i++) {
    // Een uur telt mee als het de periode raakt.
    if (weerTijd[i] + 3600 <= van || weerTijd[i] > tot) continue;
    if (weerRegen[i]) regen = true;
    if (!gradenBekend) { graden = weerGraden[i]; gradenBekend = true; }
  }
  if (!gradenBekend) return "Weer onbekend";
  if (regen) return onderweg ? "Regen onderweg - neem een jas" : "Regen op komst - neem een jas";
  return String(onderweg ? "Droog onderweg, " : "Droog, ") + String(graden) + " graden";
}

// ====================== 11. DE WEKTIJD UITREKENEN ======================

// Haalt de 3 API's op en maakt een nieuw plan. Mislukt de agenda of de trein, dan blijft
// het oude plan staan. Zo gaat de wekker ook zonder internet op de laatst bekende tijd af.
void gegevensVerversen() {
  time_t nu = time(nullptr);
  Serial.printf("API's ophalen om %s (vrij geheugen: %u bytes)\n", klokTekst(nu).c_str(), (unsigned)ESP.getFreeHeap());
  schermVerversen = true;

  weerOphalen();

  Plan nieuw;
  memset(&nieuw, 0, sizeof(nieuw));

  if (!agendaIngevuld()) {
    Serial.println("Agenda: AGENDA_LINK is nog niet ingevuld. De wekker gebruikt de vaste wektijd.");
    strcpy(nieuw.reden, "Agenda-link nog niet ingevuld");
  } else {
    Afspraak afspraak;
    if (!agendaOphalen(nu, afspraak)) return;
    nieuw.afspraak = afspraak;
```

```
    if (!afspraak.gevonden) {
      Serial.println("Agenda: geen afspraak voor vandaag of morgen.");
    } else if (!nsIngevuld()) {
      Serial.println("Trein: NS_API_SLEUTEL is nog niet ingevuld. De wekker gebruikt de vaste wektijd.");
      strcpy(nieuw.reden, "NS-sleutel nog niet ingevuld");
    } else {
      Serial.printf("Agenda: eerste afspraak %s %s\n", klokTekst(afspraak.begin).c_str(), afspraak.titel);
      time_t uiterlijk = afspraak.begin - STATION_NAAR_SCHOOL_MINUTEN * 60L;
      Reis gepland, gekozen;
      if (!treinOphalen(uiterlijk, gepland, gekozen)) return;

      if (gekozen.gevonden) {
        // Bij vertraging rekenen we met de geplande tijd: een vertraging kan weer inlopen.
        time_t vertrek     = min(gekozen.vertrek, gekozen.vertrekGepland);
        long   vooraf      = (NAAR_STATION_MINUTEN + KLAARMAAK_MINUTEN) * 60L;
        nieuw.geldig  = true;
        nieuw.trein   = gekozen;
        nieuw.deurUit = vertrek - NAAR_STATION_MINUTEN * 60L;
        nieuw.wekTijd = vertrek - vooraf;

        if (gepland.gevonden && gepland.vertrekGepland > gekozen.vertrekGepland) {
          // De trein die je normaal neemt valt uit of komt te laat aan.
          time_t      normaal = gepland.vertrekGepland - vooraf;
          const char* waarom  = gepland.valtUit ? "valt uit" : "is te laat";
          nieuw.eerderMinuten = (int)((normaal - nieuw.wekTijd) / 60);
          if (nieuw.eerderMinuten > MAX_EERDER_MINUTEN) {
            // Niet eerder wekken dan het maximum: je hebt dan minder tijd om klaar te maken.
            nieuw.wekTijd       = normaal - MAX_EERDER_MINUTEN * 60L;
            nieuw.eerderMinuten = MAX_EERDER_MINUTEN;
            snprintf(nieuw.reden, sizeof(nieuw.reden), "%s %s: haast je",
                     klokTekst(gepland.vertrekGepland).c_str(), waarom);
          } else {
            snprintf(nieuw.reden, sizeof(nieuw.reden), "%s %s: %d min eerder",
                     klokTekst(gepland.vertrekGepland).c_str(), waarom, nieuw.eerderMinuten);
          }
        }
        Serial.printf("Plan: wekker %s, deur uit %s, trein %s spoor %s%s%s\n",
                      klokTekst(nieuw.wekTijd).c_str(), klokTekst(nieuw.deurUit).c_str(),
                      klokTekst(gekozen.vertrek).c_str(), gekozen.spoor,
                      nieuw.reden[0] ? " - " : "", nieuw.reden);
      } else {
        Serial.println("Trein: geen trein gevonden die op tijd aankomt.");
        strcpy(nieuw.reden, "Geen trein gevonden");
      }
    }
  }

  plan             = nieuw;
  laatstBijgewerkt = nu;
  schermVerversen  = true;
}

// ====================== 12. SCHERM (E-INK) ======================

void knopBijhouden();   // staat in deel 14

// Het scherm verversen duurt 1 tot 4 seconden. In die tijd roept de schermbibliotheek
// steeds deze functie aan. Zo speelt het alarm door en mis je geen druk op de knop.
void schermWacht(const void* nietGebruikt) {
  (void)nietGebruikt;
  geluidVoeden();
  knopBijhouden();
}

// Breedte van een tekst in het lettertype dat nu aanstaat.
int tekstBreedte(const String& tekst) {
  int16_t  x1, y1;
  uint16_t breedte, hoogte;
  scherm.getTextBounds(tekst, 0, 0, &x1, &y1, &breedte, &hoogte);
  return breedte;
```

```
}

// Tekst waarvan de rechterkant op xRechts uitkomt.
void tekstRechts(const String& tekst, int xRechts, int y) {
  int16_t  x1, y1;
  uint16_t breedte, hoogte;
  scherm.getTextBounds(tekst, 0, y, &x1, &y1, &breedte, &hoogte);
  scherm.setCursor(xRechts - breedte - x1, y);
  scherm.print(tekst);
}

// Is het plan uit de API's nog actueel genoeg om te laten zien?
bool planTonen(time_t nu) {
  return slimAan && plan.geldig && nu < plan.afspraak.begin;
}

// Het tijdstip dat nu het belangrijkst is, voor rechtsboven op het scherm.
void belangrijksteMoment(time_t nu, String &label, String &groot) {
  if (!tijdBekend()) {
    label = "Wekker";
    groot = tijdTekst(wekUur, wekMinuut);
  } else if (toestand == SLUIMER) {
    label = "Sluimert tot";
    groot = klokTekst(sluimerTot);
  } else if (planTonen(nu) && zelfdeDag(plan.wekTijd, nu) && nu >= plan.wekTijd && nu < plan.deurUit) {
    label = "Deur uit om";
    groot = klokTekst(plan.deurUit);
  } else if (planTonen(nu) && zelfdeDag(plan.deurUit, nu) && nu >= plan.deurUit && nu < plan.trein.vertrek) {
    label = "Trein vertrekt";
    groot = klokTekst(plan.trein.vertrek);
  } else if (!wekkerAan) {
    label = "Wekker staat uit";
    groot = "";
  } else {
    time_t moment = volgendeWekMoment(nu);
    label = zelfdeDag(moment, nu) ? "Wekker vandaag" : "Wekker morgen";
    groot = klokTekst(moment);
  }
}

// De 3 regels onderaan: trein, weer en de reden of je afspraak.
void infoRegels(time_t nu, String &regel1, String &regel1Rechts, String &regel2, String &regel3) {
  regel1Rechts = "";
  if (planTonen(nu)) {
    regel1 = "Trein " + klokTekst(plan.trein.vertrek) + " spoor " + plan.trein.spoor;
    if (nu < plan.wekTijd) regel1Rechts = "deur uit " + klokTekst(plan.deurUit);
    regel2 = weerTekst(plan.deurUit, plan.afspraak.begin, true);
    if (nu - laatstBijgewerkt > 30 * 60)  regel3 = "Geen internet, gegevens van " + klokTekst(laatstBijgewerkt);
    else if (plan.reden[0])               regel3 = plan.reden;
    else                                  regel3 = klokTekst(plan.afspraak.begin) + " " + plan.afspraak.titel;
  } else {
    if (!slimAan)                   regel1 = "Slimme wektijd staat uit";
    else if (plan.reden[0])         regel1 = plan.reden;
    else if (laatstBijgewerkt > 0)  regel1 = "Geen afspraak gevonden";
    else                            regel1 = "Nog geen gegevens";
    regel2 = weerTekst(nu, nu + 3 * 3600L, false);
    regel3 = "Vaste wektijd";
    if (laatstBijgewerkt > 0) regel3 += ", bijgewerkt " + klokTekst(laatstBijgewerkt);
  }
}

// Tekent alles wat op het scherm staat.
void schermInhoud() {
  const int BREEDTE = scherm.width();    // 296
  const int HOOGTE  = scherm.height();   // 128
  const int LINKS   = 6;
  const int RECHTS  = BREEDTE - 6;

  time_t nu = time(nullptr);
  struct tm t;
  localtime_r(&nu, &t);
```

```
  scherm.fillScreen(GxEPD_WHITE);
  scherm.setTextColor(GxEPD_BLACK);
  scherm.setTextSize(1);

  // Bovenste regel: links de datum, rechts alleen een melding als de wifi weg is.
  scherm.setFont(&FreeSans9pt7b);
  if (tijdBekend()) {
    scherm.setCursor(LINKS, 15);
    scherm.print(String(DAGEN[t.tm_wday]) + " " + String(t.tm_mday) + " " + MAANDEN[t.tm_mon]);
  }
  if (WiFi.status() != WL_CONNECTED) tekstRechts("geen wifi", RECHTS, 15);

  // Links de tijd, groot. Rechts het belangrijkste tijdstip van dit moment.
  scherm.setFont(&FreeSansBold24pt7b);
  scherm.setCursor(LINKS, 60);
  scherm.print(tijdBekend() ? tijdTekst(t.tm_hour, t.tm_min) : String("--:--"));

  String label, groot;
  belangrijksteMoment(nu, label, groot);
  scherm.setFont(&FreeSans9pt7b);
  tekstRechts(label, RECHTS, 38);
  scherm.setFont(&FreeSansBold12pt7b);
  tekstRechts(groot, RECHTS, 60);

  scherm.drawFastHLine(LINKS, 68, RECHTS - LINKS, GxEPD_BLACK);

  // Onderaan: trein, weer en de reden of je afspraak.
  String regel1, regel1Rechts, regel2, regel3;
  infoRegels(nu, regel1, regel1Rechts, regel2, regel3);
  scherm.setFont(&FreeSansBold9pt7b);
  scherm.setCursor(LINKS, 85);
  scherm.print(regel1);
  int eindeRegel1 = LINKS + tekstBreedte(regel1);
  scherm.setFont(&FreeSans9pt7b);
  if (regel1Rechts.length() > 0 && RECHTS - tekstBreedte(regel1Rechts) >= eindeRegel1 + 10) {
    tekstRechts(regel1Rechts, RECHTS, 85);     // alleen als het ernaast past
  }
  scherm.setCursor(LINKS, 103);
  scherm.print(regel2);

  if (toestand == ALARM) {
    // Tijdens het alarm: een zwarte balk met wat de knop doet.
    scherm.fillRect(0, 108, BREEDTE, HOOGTE - 108, GxEPD_BLACK);
    scherm.setTextColor(GxEPD_WHITE);
    scherm.setFont(&FreeSansBold9pt7b);
    scherm.setCursor(LINKS, 123);
    scherm.print("Opstaan!");
    scherm.setFont(&FreeSans9pt7b);
    tekstRechts("kort: sluimer, lang: uit", RECHTS, 123);
    scherm.setTextColor(GxEPD_BLACK);
  } else {
    scherm.setCursor(LINKS, 122);
    scherm.print(regel3);
  }
}

// Zet de inhoud op het scherm. Meestal snel (gedeeltelijk), af en toe volledig.
void schermTekenen() {
  bool volledig = eersteTekening || deelVerversingen >= VOLLEDIG_VERVERSEN_NA;
  if (volledig) {
    scherm.setFullWindow();
    deelVerversingen = 0;
    eersteTekening   = false;
  } else {
    scherm.setPartialWindow(0, 0, scherm.width(), scherm.height());
    deelVerversingen++;
  }
  schermVerversen = false;
  scherm.firstPage();
  do {
```

```
    schermInhoud();
    geluidVoeden();
  } while (scherm.nextPage());
  scherm.powerOff();   // e-ink houdt het beeld vast zonder stroom
}

// Een korte melding tijdens het opstarten.
void schermMelding(const char* regel1, const char* regel2) {
  scherm.setFullWindow();
  scherm.firstPage();
  do {
    scherm.fillScreen(GxEPD_WHITE);
    scherm.setTextColor(GxEPD_BLACK);
    scherm.setFont(&FreeSansBold9pt7b);
    scherm.setCursor(6, 56);
    scherm.print(regel1);
    scherm.setFont(&FreeSans9pt7b);
    scherm.setCursor(6, 82);
    scherm.print(regel2);
  } while (scherm.nextPage());
  scherm.powerOff();
}

void schermStarten() {
  scherm.init(0);
  scherm.epd2.setBusyCallback(schermWacht);
  scherm.setRotation(1);            // liggend: 296 breed, 128 hoog
  scherm.setTextWrap(false);
  Serial.println("Scherm klaar.");
}

// ====================== 13. HET ALARM ======================

void alarmStarten() {
  toestand        = ALARM;
  alarmGestartOp  = millis();
  sluimerTot      = 0;
  testBezig       = false;
  schermVerversen = true;
  geluidStarten(gekozenGeluid, VOLUME_START);
  Serial.printf("WEKKER! Alarm gaat af met geluid %s.\n", GELUIDEN[gekozenGeluid].naam);
}

void alarmStoppen(const char* reden) {
  geluidStoppen();
  toestand        = RUST;
  sluimerTot      = 0;
  schermVerversen = true;
  Serial.printf("Alarm uit (%s).\n", reden);
}

void alarmSluimeren(const char* reden) {
  geluidStoppen();
  toestand        = SLUIMER;
  sluimerTot      = time(nullptr) + SLUIMER_MINUTEN * 60L;
  schermVerversen = true;
  Serial.printf("Sluimeren tot %s (%s).\n", klokTekst(sluimerTot).c_str(), reden);
}

// Kijkt of het alarm moet beginnen, harder moet of vanzelf moet stoppen.
void alarmBijwerken() {
  time_t nu = time(nullptr);

  if (toestand == RUST) {
    if (wekkerAan && tijdBekend() && nu >= volgendeWekMoment(nu)) {
      struct tm t;
      localtime_r(&nu, &t);
      laatsteAlarmDag = t.tm_yday;   // vandaag niet nog een keer
      alarmStarten();
    }
  } else if (toestand == SLUIMER) {
```

```
    if (nu >= sluimerTot) alarmStarten();
  } else {
    // Het alarm speelt: volume langzaam opbouwen.
    unsigned long seconden = (millis() - alarmGestartOp) / 1000;
    if (seconden >= (unsigned long)VOLUME_OPBOUW_SECONDEN) {
      geluidVolume = VOLUME_MAX;
    } else {
      geluidVolume = VOLUME_START + (VOLUME_MAX - VOLUME_START) * (int)seconden / VOLUME_OPBOUW_SECONDEN;
    }
    if (seconden >= (unsigned long)ALARM_MAX_MINUTEN * 60) alarmStoppen("te lang afgegaan");
  }
}

// Speelt het gekozen geluid 5 seconden af (knop "Test geluid" op de webpagina).
void geluidTesten() {
  if (toestand == ALARM) return;
  geluidStarten(gekozenGeluid, (VOLUME_START + VOLUME_MAX) / 2);
  testBezig     = true;
  testGestartOp = millis();
  Serial.printf("Test: geluid %s, 5 seconden.\n", GELUIDEN[gekozenGeluid].naam);
}

// ====================== 14. DE KNOP ======================

// De knop zit op A0. A0 meet een spanning: los is dat ongeveer 0, ingedrukt (verbonden
// met 3V) ongeveer 1023. Deze functie wordt heel vaak aangeroepen en onthoudt een druk
// op de knop tot knopAfhandelen() hem ophaalt.
void knopBijhouden() {
  static unsigned long laatsteMeting  = 0;
  static unsigned long ingedruktSinds = 0;
  static bool          wasIngedrukt   = false;
  static bool          langGemeld     = false;

  unsigned long nu = millis();
  if (nu - laatsteMeting < 30) return;     // niet vaker dan 30 keer per seconde meten (anders stoort het de wifi)
  laatsteMeting = nu;

  bool ingedrukt = analogRead(PIN_KNOP) > KNOP_DREMPEL;
  if (ingedrukt && !wasIngedrukt) {
    ingedruktSinds = nu;
    langGemeld     = false;
  }
  if (ingedrukt && !langGemeld && nu - ingedruktSinds >= (unsigned long)KNOP_LANG_MS) {
    langGemeld     = true;
    knopActieKlaar = KNOP_LANG;            // 1 seconde vastgehouden
  }
  if (!ingedrukt && wasIngedrukt && !langGemeld) {
    knopActieKlaar = KNOP_KORT;            // kort gedrukt en losgelaten
  }
  wasIngedrukt = ingedrukt;
}

void knopAfhandelen() {
  knopBijhouden();
  KnopActie actie = knopActieKlaar;
  knopActieKlaar  = KNOP_NIETS;
  if (actie == KNOP_NIETS) return;

  if (toestand == ALARM) {
    if (actie == KNOP_KORT) alarmSluimeren("knop");
    else                    alarmStoppen("knop");
  } else if (toestand == SLUIMER && actie == KNOP_LANG) {
    alarmStoppen("knop");
  } else if (testBezig) {
    geluidStoppen();
    testBezig = false;
  }
}

// ====================== 15. WIFI EN DE WEBPAGINA ======================
```

```
void wifiStarten() {
  WiFi.mode(WIFI_STA);
  WiFi.hostname(NAAM_OP_NETWERK);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_NAAM, WIFI_WACHTWOORD);
  Serial.printf("Verbinden met wifi \"%s\"", WIFI_NAAM);
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 20000) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("LET OP: geen wifi. Klopt de naam en het wachtwoord bovenaan de code?");
    Serial.println("        De ESP8266 werkt alleen op 2,4 GHz. De wekker blijft het proberen.");
  }
}

// Meldt het als de wifi wegvalt of terugkomt, en probeert opnieuw te verbinden.
void wifiBewaken() {
  bool verbonden = WiFi.status() == WL_CONNECTED;
  if (verbonden && !wifiWasVerbonden) {
    Serial.print("Wifi verbonden. Open in je browser: http://");
    Serial.print(WiFi.localIP());
    Serial.printf("  (of http://%s.local)\n", NAAM_OP_NETWERK);
    MDNS.close();
    if (MDNS.begin(NAAM_OP_NETWERK)) MDNS.addService("http", "tcp", 80);
    schermVerversen = true;
    verversNu       = true;
  }
  if (!verbonden && wifiWasVerbonden) {
    Serial.println("Wifi weggevallen. De klok loopt door, de webpagina werkt even niet.");
    schermVerversen = true;
  }
  if (!verbonden && millis() - laatsteWifiPoging > 30000) {
    laatsteWifiPoging = millis();
    WiFi.reconnect();
  }
  if (verbonden) MDNS.update();
  wifiWasVerbonden = verbonden;
}

// Maakt tekst veilig om in de webpagina te zetten.
String htmlVeilig(const String& tekst) {
  String uit = tekst;
  uit.replace("&", "&amp;");
  uit.replace("<", "&lt;");
  uit.replace(">", "&gt;");
  return uit;
}

// De webpagina waarop je de wekker instelt.
void paginaTonen() {
  time_t nu = time(nullptr);
  struct tm t;
  localtime_r(&nu, &t);

  String html;
  html.reserve(3600);
  html += F("<!DOCTYPE html><html lang='nl'><head><meta charset='utf-8'>"
            "<meta name='viewport' content='width=device-width, initial-scale=1'>"
            "<title>Slimme wekker</title><style>"
            "body{font-family:sans-serif;max-width:420px;margin:24px auto;padding:0 16px;color:#111}"
            "h1{color:#FF8A00;margin-bottom:4px}"
            ".status{background:#f2f2f2;border-radius:10px;padding:12px 14px;margin:12px 0;line-height:1.5}"
            "label{display:block;margin:16px 0 6px;font-weight:bold}"
            "input[type=time],select{font-size:20px;padding:8px;width:100%;box-sizing:border-box}"
            ".vink{font-weight:normal;font-size:18px}"
            "button{font-size:17px;padding:12px 22px;border:0;border-radius:24px;background:#FF8A00;"
            "color:#fff;margin:14px 8px 0 0}"
            "button.grijs{background:#555}form{display:inline}"
```

```
            "</style></head><body><h1>Slimme wekker</h1>");

  // Wat de wekker nu weet
  html += F("<div class='status'>");
  if (tijdBekend()) {
    html += "Het is nu <b>" + tijdTekst(t.tm_hour, t.tm_min) + "</b>, ";
    html += String(DAGEN[t.tm_wday]) + " " + String(t.tm_mday) + " " + MAANDEN[t.tm_mon] + ".<br>";
    String label, groot;
    belangrijksteMoment(nu, label, groot);
    if (toestand == ALARM) html += F("<b>Het alarm gaat nu af.</b><br>");
    else                   html += "<b>" + label + " " + groot + "</b><br>";
    String regel1, regel1Rechts, regel2, regel3;
    infoRegels(nu, regel1, regel1Rechts, regel2, regel3);
    html += htmlVeilig(regel1);
    if (regel1Rechts.length() > 0) html += ", " + regel1Rechts;
    html += "<br>" + regel2 + "<br>" + htmlVeilig(regel3);
    if (planTonen(nu)) html += "<br>Bijgewerkt " + klokTekst(laatstBijgewerkt);
  } else {
    html += F("De tijd is nog niet opgehaald.");
  }
  html += F("</div>");

  html += F("<form method='post' action='/opslaan' style='display:block'>"
            "<label for='tijd'>Vaste wektijd</label><input type='time' id='tijd' name='tijd' value='");
  html += tijdTekst(wekUur, wekMinuut);
  html += F("' required><label for='geluid'>Alarmgeluid</label><select id='geluid' name='geluid'>");
  for (int i = 0; i < AANTAL_GELUIDEN; i++) {
    html += "<option value='" + String(i) + "'";
    if (i == gekozenGeluid) html += " selected";
    html += ">" + String(GELUIDEN[i].naam) + "</option>";
  }
  html += F("</select><label class='vink'><input type='checkbox' name='aan'");
  if (wekkerAan) html += F(" checked");
  html += F("> Wekker aan</label><label class='vink'><input type='checkbox' name='slim'");
  if (slimAan) html += F(" checked");
  html += F("> Slimme wektijd (agenda en trein)</label><button type='submit'>Opslaan</button></form>");

  html += F("<form method='post' action='/ververs'><button class='grijs'>API's ophalen</button></form>");
  html += F("<form method='post' action='/test'><button class='grijs'>Test geluid</button></form>");
  if (toestand == ALARM) {
    html += F("<form method='post' action='/sluimer'><button class='grijs'>Sluimeren</button></form>");
  }
  if (toestand != RUST) {
    html += F("<form method='post' action='/stop'><button class='grijs'>Alarm uit</button></form>");
  }
  html += F("</body></html>");

  server.send(200, "text/html; charset=utf-8", html);
}

void terugNaarPagina() {
  server.sendHeader("Location", "/");
  server.send(303, "text/plain", "");
}

void paginaOpslaan() {
  String tijd = server.arg("tijd");   // bijvoorbeeld "07:30"
  int uur    = tijd.substring(0, 2).toInt();
  int minuut = tijd.substring(3, 5).toInt();
  if (tijd.length() >= 5 && tijd.charAt(2) == ':' && uur >= 0 && uur <= 23 && minuut >= 0 && minuut <= 59) {
    if (uur != wekUur || minuut != wekMinuut) laatsteAlarmDag = -1;   // nieuwe tijd mag vandaag nog afgaan
    wekUur    = uur;
    wekMinuut = minuut;
  } else {
    Serial.printf("Webpagina: wektijd \"%s\" klopt niet, de oude tijd blijft staan.\n", tijd.c_str());
  }
  int geluid = server.arg("geluid").toInt();
  if (geluid >= 0 && geluid < AANTAL_GELUIDEN) gekozenGeluid = geluid;
  wekkerAan = server.hasArg("aan");
  slimAan   = server.hasArg("slim");
```

```
  instellingenBewaren();
  schermVerversen = true;
  terugNaarPagina();
}

void paginaVervers() {
  verversNu = true;      // loop() haalt de API's op, dat duurt een paar seconden
  terugNaarPagina();
}

void paginaTest() {
  geluidTesten();
  terugNaarPagina();
}

void paginaSluimer() {
  if (toestand == ALARM) alarmSluimeren("webpagina");
  terugNaarPagina();
}

void paginaStop() {
  if (toestand != RUST) alarmStoppen("webpagina");
  terugNaarPagina();
}

void paginaNietGevonden() {
  server.send(404, "text/plain; charset=utf-8", "Deze pagina bestaat niet. Ga naar / voor de wekker.");
}

void webserverStarten() {
  server.on("/", HTTP_GET, paginaTonen);
  server.on("/opslaan", HTTP_POST, paginaOpslaan);
  server.on("/ververs", HTTP_POST, paginaVervers);
  server.on("/test", HTTP_POST, paginaTest);
  server.on("/sluimer", HTTP_POST, paginaSluimer);
  server.on("/stop", HTTP_POST, paginaStop);
  server.onNotFound(paginaNietGevonden);
  server.begin();
}

// ====================== 16. SETUP EN LOOP ======================

void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println();
  Serial.println();
  Serial.println("=== Slimme wekker start op ===");

  memset(&plan, 0, sizeof(plan));
  instellingenLaden();
  geluidKlaarzetten();
  schermStarten();

  schermMelding("Slimme wekker", "Verbinden met wifi...");
  wifiStarten();
  configTime(TIJDZONE, TIJDSERVER_1, TIJDSERVER_2);
  webserverStarten();

  // Even wachten op de tijd, zodat het scherm meteen goed staat.
  unsigned long start = millis();
  while (WiFi.status() == WL_CONNECTED && !tijdBekend() && millis() - start < 10000) delay(100);
  if (WiFi.status() == WL_CONNECTED && !tijdBekend()) {
    Serial.println("LET OP: de tijd is nog niet opgehaald. De wekker probeert het vanzelf opnieuw.");
  }
}

void loop() {
  geluidVoeden();
  server.handleClient();
  geluidVoeden();
```

```
  wifiBewaken();
  knopAfhandelen();
  alarmBijwerken();

  // Testgeluid na 5 seconden stoppen.
  if (testBezig && millis() - testGestartOp >= (unsigned long)TEST_DUUR_MS) {
    testBezig = false;
    if (toestand != ALARM) geluidStoppen();
  }

  // Tijd net binnen? Meld het 1 keer.
  if (tijdBekend() && !tijdWasBekend) {
    tijdWasBekend = true;
    time_t nu = time(nullptr);
    struct tm t;
    localtime_r(&nu, &t);
    Serial.printf("Tijd opgehaald: %s, %s %d %s.\n", tijdTekst(t.tm_hour, t.tm_min).c_str(),
                  DAGEN[t.tm_wday], t.tm_mday, MAANDEN[t.tm_mon]);
    schermVerversen = true;
  }

  // API's ophalen: bij het opstarten, daarna elke VERVERS_MINUTEN. Dat duurt een paar seconden,
  // dus niet terwijl het alarm afgaat, je sluimert of een testgeluid speelt.
  bool tijdOm = millis() - laatsteVerversPoging >= (unsigned long)VERVERS_MINUTEN * 60000UL;
  if ((verversNu || tijdOm) && toestand == RUST && !testBezig && tijdBekend() && WiFi.status() == WL_CONNECTED) {
    verversNu            = false;
    laatsteVerversPoging = millis();
    gegevensVerversen();
  }

  // Scherm: 1 keer per minuut, of als er iets veranderd is.
  time_t nu = time(nullptr);
  struct tm t;
  localtime_r(&nu, &t);
  if (t.tm_min != vorigeMinuut) {
    vorigeMinuut    = t.tm_min;
    schermVerversen = true;
  }
  if (schermVerversen) schermTekenen();

  geluidVoeden();
  delay(1);
}
```